

UNIVERSITY OF DERBY

School of Computing

MSc Information Technology

**Web-based IDE Environment for Concurrent
Programming in Graphical Language of
Place-Transition Nets**

A Dissertation Submitted in Partial Fulfilment of the Requirements
for the Degree of Master of Science in Information Technology

Author: Manav Manav

Student ID: 100799466

Supervisor: Dr. Dmitry Zaitsev

Module: 7CS997 Independent Studies

Academic Year: 2025–2026

Abstract

Petri nets are used to represent and analyze systems involving simultaneity. A pioneer in the field, Carl Adam Petri, introduced Petri nets in 1962. Since then Petri nets have become sought after formal tools in computer science and engineering. TINA, the first major Petri net toolkit, was developed as a desktop application at LAAS-CNRS in France. Though TINA is the most powerful tool, it is not beginner friendly. Researchers and students would have to spend significant time installing and configuring TINA to work with it.

The objective of this thesis is to develop a user friendly tool to develop Place-Transition Petri nets. The tool is a browser based IDE which allows users to develop Petri nets without the hassles of installations. Users of the tool can develop a Petri net by drawing places, transitions, and arcs, manage tokens with their weights, develop inhibitor arcs, and run the net through simulation using either step-by-step or fully automated simulation.

The tool is written using JavaScript with 5 primary modules. Four of these modules work with browser APIs, whereas the fifth module, engine.js, include the simulation logic of the tool and thus can be run standalone using Node.js. The back end Rest API built with Node.js uses Express for naming and saving net definitions for browser sessions.

The tool allows you to import and export TINA's native NDR file format. The NDR parser can handle all variants of the published formats by the supervisor, including curved arc notation, visually routed arcs, braced mathematical place holders such as $\{z=x+y\}$, and $\{Y \div 2\}$, and inhibitor arc syntax. Three reference models from the supervisor's collection were imported successfully: add.ndr (5 places, 3 transitions, 11 arcs, includes 5 inhibitor arcs), fdiv.ndr (48 places, 61 transitions, 225 arcs, includes 65 inhibitor arcs) and fmul.ndr (15 places, 16 transitions, 65 arcs, includes 21 inhibitor arcs). All three models were imported without any broken connections.

When TINA's evaluation is applied, complete NDR format, and structural correctness is confirmed. This tool will fill a gap in the browser based IDE's capability, which previously lacked this level of TINA support.

Keywords: Petri Nets, Place-Transition Nets, Web-based IDE, Inhibitor Arcs, NDR Format, TINA Compatibility, JavaScript, REST API, Simulation Engine, Modular Architecture

Acknowledgements

To Dr Dmitry Zaitsev, I would like to say thank you for your hard work and patience in your supervision during this project. Your extensive knowledge of Petri net theory shaped the technical aspects of the tool at each step. It was you that advised against PNML and favored NDR as the preferred file format. Your preference was to represent inhibitor arcs as weight zero, and that a description field should be provided for places. It was your preference to include curved bezier arcs. These suggestions all of these were your preference, and they were all suggest during our fortnightly meetings.

Your published research, and in particular your collections of models, were a valuable and formal test set for the NDR parser. Having your own published models available to me in the browser-based editor and able to check the structural correctness of your models really added value to the evaluation chapter.

I thank the University Of Derby School Of Computing for the excellent academic climate and resources that enabled this project.

List of Abbreviations

Abbreviation	Full Meaning
IDE	Integrated Development Environment
NDR	Net Description and Representation (TINA native graphical format)
PNML	Petri Net Markup Language (ISO/IEC 15909-2 standard)
TINA	TIme petri Net Analyser (LAAS-CNRS, France)
API	Application Programming Interface
REST	Representational State Transfer
SVG	Scalable Vector Graphics
JSON	JavaScript Object Notation
DOM	Document Object Model
DSR	Design Science Research
UI	User Interface
PT net	Place-Transition Petri Net (untimed, uncoloured)
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets

Table of Contents

Abstract	2
Acknowledgements	3
List of Abbreviations	4
Table of Contents	5
Chapter 1: Introduction.....	7
1.1 Research Background and Context.....	7
1.2 Motivation and Problem Statement	8
1.3 Research Aim and Objectives.....	9
1.4 Research Questions	9
1.5 Scope and Delimitations.....	10
1.6 Dissertation Structure	10
Chapter 2: Literature Review	11
2.1 Introduction	11
2.2 Petri Net Theory — Formal Foundations	11
2.2.1 Historical Development.....	11
2.2.2 Key Behavioural Properties.....	11
2.2.3 Inhibitor Arcs.....	12
2.3 Existing Petri Net Tools	12
2.4 File Formats for Petri Net Exchange	13
2.4.1 PNML — The ISO Standard	13
2.4.2 NDR — The TINA Native Format	13
2.4.3 Format Selection Rationale.....	13
2.5 Research Gap and Project Positioning.....	14
Chapter 3: Research Methodology	15
3.1 Research Philosophy and Approach	15
3.2 Iterative Supervisor-Guided Development	15
3.3 Evaluation Framework	16
Chapter 4: System Design and Implementation	17
4.1 Architecture Overview	17
4.2 The Simulation Engine (engine.js)	17
4.2.1 The isEnabled() Function	18
4.2.2 The fireTransition() Function	19
4.3 The Renderer (renderer.js).....	20
4.3.1 SVG-Based Rendering	20
4.3.2 The borderPt() Function — Arc Endpoint Calculation.....	20
4.3.3 The bezierPath() Function — Curved Arcs	20
4.4 Inhibitor Arcs — Implementation and Rendering	21

4.5 The NDR File Format Parser	22
4.5.1 Tokenisation — Handling Braced Labels	23
4.5.2 Curved Arc Detection — The <code>_isFloat()</code> Check	24
4.5.3 Place Label Extraction	24
4.6 NDR Export	25
4.7 The Graphical Editor	26
4.7.1 Drawing Tools and Interaction	26
4.7.2 Click vs Drag — Disambiguation	27
4.8 Automatic Simulation	28
4.9 Back-end REST API	28
Chapter 5: Testing and Evaluation	30
5.1 Feature Testing	30
5.2 NDR Compatibility Evaluation	31
5.2.1 <code>add.ndr</code> — Adder Circuit	31
5.2.2 <code>fdiv.ndr</code> — Integer Division Algorithm	31
5.2.3 <code>fmul.ndr</code> — Multiplication Algorithm	32
5.3 Evaluation Results	33
5.4 Feature Comparison with TINA	34
Chapter 6: Discussion	35
6.1 Achievement of Research Objectives	35
6.2 Technical Discussion	35
6.2.1 The Separation of <code>engine.js</code> from the UI	35
6.2.2 The Weight-0 Inhibitor Arc Convention	35
6.3 Limitations	36
6.4 Relationship to the Literature	36
Chapter 7: Conclusions and Recommendations	37
7.1 Summary of Contributions	37
7.2 Recommendations for Future Work	37
7.2.1 PNML Export	37
7.2.2 NDR Import Scaling	37
7.2.3 Undo/Redo	37
7.2.4 Server-Side Formal Analysis	37
7.2.5 Migration to a Component Framework	38
7.3 Final Conclusion	38
References	39
Appendix A: Project File Structure	41
Appendix B: Complete NDR Format Reference	42
Appendix C: Keyboard Shortcuts Reference	43

Chapter 1: Introduction

1.1 Research Background and Context

Carl Adam Petri developed Petri nets in his doctorate dissertation completed in 1962 at the University of Bonn (Petri, 1962). Petri recognized the need for a formal language to address causality and concurrency in the behavior of computer systems. Petri's invention was an extremely general model based on places, transitions, arcs, and tokens. Six decades since its development, the Petri net formalism was used in software engineering and communications protocols, workflow control, business processes, biological systems, and other safety-critical systems (Murata, 1989; Reisig, 2013).

The main strength of Petri nets is their combination of an elegant graphical notation and theory foundation. The graphical notation simplifies understanding and explaining models to non-experts. The mathematical theory provides a foundation for analysis and reasoning about system behavior. Reachability, boundedness, liveness, and deadlock analysis of Petri nets are described in detail in Murata's (1989) work, which continues to be the main reference for Petri net theories.

A Place-Transition Petri net is denoted by $PN = (P, T, F, G, M_0)$, where P is a finite non-empty set of places, T is a finite non-empty set of transitions, $P \cap T = \emptyset$, $F: (P \times T) \rightarrow \mathbb{N}$ states the weight of each input arc and how many tokens it requires, $G: (T \times P) \rightarrow \mathbb{N}$ states the weight of each output arc and how many tokens it produces, and $M_0: P \rightarrow \mathbb{N}$ is the initial marking.

A transition t is enabled under marking M if and only if $M(p) \geq F(p, t)$ for all input places p in the preset of t . When t fires, the marking changes to the new marking M' defined by $M'(p) = M(p) - F(p, t) + G(t, p)$ for every place p . This is the firing rule that controls all token dynamics in the system and is the primary rule that is implemented in the engine.js module of this project.

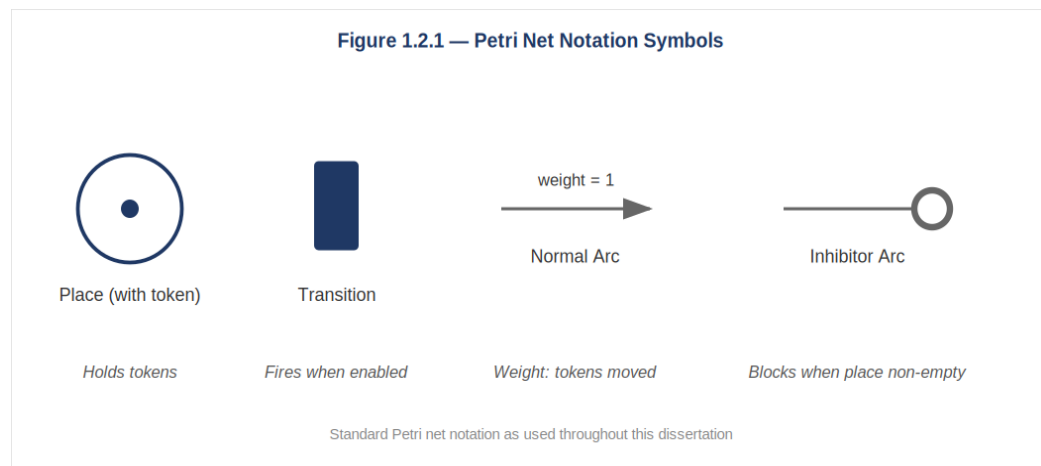


Figure 1.1: Standard Petri net notation. Circles (places) hold tokens (black dots). Rectangles (transitions) fire when enabled. Normal arcs have arrowheads; inhibitor arcs end in a circle, requiring the connected place to be empty.

Inhibitor arcs are used to enhance the basic formalism. An inhibitor arc from place p to transition t contains the condition $M(p) = 0$ in the firing rule for t . This means that a place cannot be occupied by any tokens for the transition to fire. This additional ability allows models to incorporate absence as a required condition, which cannot be achieved by regular arcs. Engelfriet

(1991) demonstrated that introducing inhibitor arcs in Petri nets increases their computational power to Turing completeness.

Simple example of how the basic notation and the firing rule work with a three-place, two-transition Petri net shown in figure 1.2.

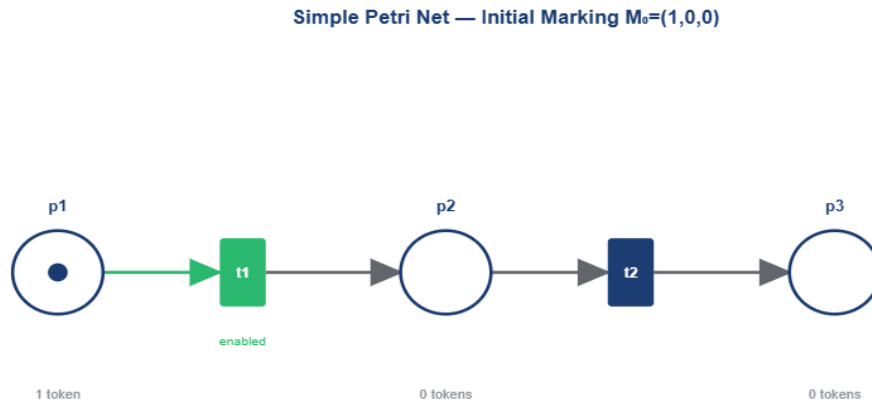


Figure 1.2: Simple Petri net with initial marking $M_0 = (1, 0, 0)$. Transition $t1$ is enabled (green) because place $p1$ holds one token. When $t1$ fires, the marking changes to $M_1 = (0, 1, 0)$.

1.2 Motivation and Problem Statement

The primary GUI-based software for work containing Petri nets is the TINA (Time petri Net Analyser) toolbox. The VASY team of LAAS-CNRS in Toulouse, France, created and maintains TINA. It includes a Petri net editor and a step-wise simulation as well as sophisticated tools for formal analysis and the construction of reachability graphs, liveness checking, detection of deadlocks, and verification of properties of linear logic. TINA supports the NDR and PNML formats for processing graphical nets.

Although the technical details of TINA are substantial, accessibility is a major concern. It is a desktop tool. Thus, having TINA means that one has to download, install, and configure a tool for one's own operating system and learn the tool interface that was primarily designed by Petri net specialists. For a beginner postgraduate or researcher who tries to construct a model quickly and lacks a formal tool installation, the barrier is significant.

No browser based alternative exists, and a major tool surfacing investigation at the beginning of this work showed that each of the major Petri Net tools (TINA, WoPeD, GreatSPN, Pipe2, PetriSim, etc.) are desktop tools. There does not exist a formally correct browser-based IDE for Place-Transition Petri nets that a student or researcher can use after just a URL.

This project fulfills that need. The aim is not to substitute for TINA. TINA's formal analysis capabilities are greatly beyond what is possible in a browser-based tool in the given length of an MSc project. However, this project aims to offer a free and easy-to-use browser-based tool to work with Petri nets that is consistent with TINA.

1.3 Research Aim and Objectives

This study aims to develop a web-based Integrated Development Environment for Place-Transition Petri nets that is correct by design, easy to use, and directly compatible with the TINA NDR format.

The following objectives have been set for this study:

- A comprehensive analysis of existing browser-based editors for Petri nets will be carried out.
- The development of a web-based graphical Place-Transition Petri net editor.
- The development of a formal Petri net simulation engine that supports all inhibitor arc cases.
- Support for the export and import of the TINA NDR format (including the variant of curved arcs).
- A Node.js / Express back-end REST API will be created to persistently store and retrieve named nets across browser sessions.
- This tool will be evaluated against TINA by importing and verifying three models that have been published in the supervisor's research.

1.4 Research Questions

- Can a correctly formatted Place-Transition Petri net Simulator be built to directly run in a web browser without the need for installation?
- When parsing the TINA NDR file format, what is the best approach to deal with variants of the format, curved arc notation, and math labels?
- What is the simulation engine and UI software architecture that allows the engine to be completely tested independent of the UI and accessible to pre-build the UI for future server-side deployment?
- How does the web-based tool compare to TINA for teaching Petri Nets within the available features for learning and teaching?

1.5 Scope and Delimitations

The project will examine the aspects of untimed, uncoloured Place-Transition Petri nets, the variant of Petri nets introduced by Murata (1989). It will not examine Timed Petri nets, Coloured Petri nets or other complex Petri net variants, and considers these to be future work.

The tool does not aim to copy TINA's formal analysis tools (reachability graph computation, liveness, deadlock, and invariant checking). These tools are complicated and require significant computational and mathematical tools that are outside the boundaries of an MSc project focused on an editor design. This tool aims for a graphical editor, token simulation, and improved file format support.

1.6 Dissertation Structure

Chapter 2 provides relevant literature on Petri net theory and other existing tools and file formats. Chapter 3 describes the research methodology of the iterative, supervisor-guided development process and the evaluation framework. Chapter 4 is the main technical chapter describing the system architecture and the implementation of each module in detail, with code extracts. Chapter 5 describes the testing and evaluation and includes the NDR model import tests. Chapter 6 discusses the results, limitations, and the comparison with existing work. Chapter 7 is a summary of the findings and suggestions for future work.

Chapter 2: Literature Review

2.1 Introduction

This chapter outlines some of the Petri net theories and provides information on existing tools and formats used to share data to and from tools. It describes the gap in existing research and explains the decisions made during this project for which technical solutions were required.

As Petri nets were first introduced for the analysis of concurrent systems and later were applied to many other domains, theoretical foundations spanning several disciplines have been developed (see, e.g., Preparedness and response systems, 1998). Although the underlying theory is far from complete and new research continues in this direction, many applications have been developed for a variety of use cases.

2.2 Petri Net Theory — Formal Foundations

2.2.1 Historical Development

A main part of Petri's doctorate studies was the development of Petri nets. He developed them to represent causal relationships of processes in parallel computing systems. The definition of Petri nets became more formalized through the years of the 1970s and 1980s. Murata's survey (1989), which was a landmark piece, brought these definitions and studies into the core of computer science. It was the first survey to give an overview of the most fundamental behaviors and techniques for the algebraic analysis of Petri nets which are still in use.

The Petri Nets conference, which began in 1980, is the main venue for advances in the theory of Petri nets. The field has grown to deal with many different extensions of the basic model. Each of these extensions increases the model's ability to express computational systems. However, most extensions cause the model to become less tractable.

2.2.2 Key Behavioural Properties

Murata (1989) established four basic behavioral characteristics of Petri nets. The first of these is reachability and asks whether from initial marking M_0 there exists a series of transitions such that marking M' is obtained. This is the basis of functional verification. Boundedness checks if there is a limit on the number of tokens in every place within all reachable markings. An unbounded net describes a system with unbounded queues or unbounded memory. Liveness asks if there is a limit on the number of markings after which each of the net's transitions will have been executed. A non-live net has disabled transitions, which may contain a design error. Safeness is a special case of boundedness where every place is limited by one token.

These properties are decidable for bounded nets, but, in the general case, they are undecidable, as demonstrated by Hack (1975). It is, hence, justified and correct to limit formal analysis to bounded nets, and the tools that are available, including TINA, ask for a bounded state-space size to be supplied prior to computation of reachability graphs.

2.2.3 Inhibitor Arcs

Engelfriet (1991), in one of the first studies on inhibitor arcs, explained the zero-testing capability extension of Petri nets by the addition of inhibitor arcs. He outlined the formal semantics of inhibitor arcs and stressed that they provide Place/Transition nets with full computation power, i.e., the ability to realize anything computed by a Turing machine.

In the original notation, an inhibitor arc is drawn with a dot instead of an arrowhead. This notation has been adopted most widely in the literature (Murata, 1989; Engelfriet, 1991) and in the TINA tool and has been used in the notation in this dissertation as well.

2.3 Existing Petri Net Tools

Several software tools have been developed for working with Petri nets. Table 2.1 surveys the most widely used:

Tool	Developer	Type	Key Features	NDR Support	Limitations
TINA	LAAS-CNRS, France	Desktop	Full analysis, simulation, NDR, PNML	Native	Requires installation; complex UI
WoPeD	University of Eindhoven	Desktop	Workflow nets, PNML, educational	No	Limited to workflow nets only
GreatSPN	University of Turin	Desktop	Stochastic, GSPN analysis, PNML	No	Specialist focus; complex setup
Pipe2	Imperial College London	Desktop	PNML, basic simulation, Java-based	No	Outdated; limited features
INA	Humboldt University	Desktop	Invariant analysis, reachability	No	Command-line interface only
This project	University of Derby	Browser	PT nets, NDR import/export, no install	Full import+export	No formal analysis yet

Two things are notable about the survey. First, every existing tool is a desktop application, none of them run in a web browser. Second, TINA is the only tool that supports the native NDR file format among the reviewed tools.

This project will be the first browser-based tool that completely supports the NDR file format.

2.4 File Formats for Petri Net Exchange

2.4.1 PNML — The ISO Standard

Partially translated from XML, Petri Net Markup Language (PNML) is an ISO/IEC 15909-2 standard and is capable of marking up places, transitions, arcs, and layout information (Kindler, 2006). XML's hierarchical and modular nature makes it fairly easy to mark up languages. As an international standard, PNML is supported by most Petri net tools, including TINA, WoPeD, GreatSPN, and Pipe2. The problem, however, is that PNML XML's verbose nature makes the markup of even the most basic 3-place, 2-transition net a burden, with namespaces, schemas and even meta-graphical elements needing to be declared and marked up across several lines.

2.4.2 NDR — The TINA Native Format

NDR is the native graphical format of the TINA toolbox, and is explained in the official TINA format manual (LAAS-CNRS, 2024). This is a simple text format. Each line contains one space separated declaration. The grammar is compact:

```
p x y name marking anchor [label anchor]      - place declaration
t x y name [anchor] eft lft anchor [{label} anchor] - transition
e from to weight anchor                        - simple arc
e from f1 f2 to f3 f4 weight anchor            - curved arc
e from to ?-k anchor                          - inhibitor arc
h netname                                     - net name (last line, mandatory)
```

There are two formats for NDR arcs. The simple format contains the names of the source and target, weight and anchor and is written in a line. The curved format is used when TINA lays out arcs around other elements for readability. This requires two floating point numbers for attachment coordinates to be written between the source and target names. In this project, the main technical challenge in the design of the NDR parser was correctly detecting and interpreting this curved format.

For inhibitor arcs, in TINA, the notation $?-k$ is used, where k is the threshold: the arc blocks the transition if the connected place has k or more tokens. When k is set to 1, this notation describes the behavior of an inhibitor arc, i.e., an arc will be inhibitor if the place where it connects to is empty. This notation is identical to the weight-0 notation used in this project; every inhibitor arc is exported as $?-1$ in this project.

2.4.3 Format Selection Rationale

The supervisor clearly endorsed NDR over PNML for this project for two reasons: first, in NDR generation from code, there are no unnecessary details. Second, NDR files can be opened in TINA's nd graphical editor, which allows for a direct comparison between the tools. Having accepted this recommendation, we proceeded to use NDR as our main format for exchange. Work on the generation of PNML is scheduled for later.

2.5 Research Gap and Project Positioning

This chapter presents tools and formats. A clear gap exists. At the beginning of this project, no fully functional Petri net IDE existed as a web app without the need for installation that met all the formal criteria. The established tools are desktop applications.

Moreover, NDR is the native format of the most well-known Petri net tool (TINA). Before this project, there was no web application and no full support for NDR import and export.

This project fills that gap. This tool does not aim to compete with TINA because TINA provides formal analysis that goes way beyond what is achievable in the scope of a MSc project in a web browser. This tool aims to be the first correct and easily accessible step. This tool is open to use directly in all browsers, can be used for pedagogy, and can allow the community to engage in the exchange of files with TINA for more advanced analyses.

Chapter 3: Research Methodology

3.1 Research Philosophy and Approach

This research study relies on Peffers et al (2020) for a description of the Design Science Research (DSR) style of research. DSR is particularly suitable for software engineering research because it is based on the development and assessment of artifacts (models, tools, and systems), where these artifacts are the main contributions in the research. In DSR, knowledge is constructed through the activity of building something that works and making it accessible. This is contrary to traditional empirical research, in which knowledge is constructed through the observation of existing phenomena.

The DSR methodology was followed in six main steps: Defining the problem (there is no browser-based Petri net IDE with NDR support), establishing aims (browser-based, correct by formality, NDR support, TINA interoperability), constructing the artifact (the editor, simulator, and NDR tools), showing its capability (with supervisor's research models), assessing it (zero broken connections, feature parity with TINA), and communicating the outcome (this dissertation).

3.2 Iterative Supervisor-Guided Development

The tool was built across 4 iterative stages. Each stage was driven by fortnightly supervisor meetings. This meant every tool feature could be attributed directly to a supervisor request, guiding the development and ensuring it remained on task. The requests and suggestions from the supervisor were not primarily of a supervisory nature. The supervisor, as the primary user of the tool, was acting as the domain expert guiding the various stages of the tool design.

Figure 3.1 shows the four development stages and their key deliverables:



Figure 3.1: Four iterative development stages. Each stage was presented at a fortnightly supervisor meeting, and the next stage's requirements came from that meeting's discussion.

Stage	Meeting Date	Key Features	Supervisor Input
1 — Basic Editor	16 June 2026	Places, transitions, arcs, drag-and-drop, token tool, erase	Confirmed approach; requested arc weight editing next
2 — Properties	23 June 2026	Properties panel: rename, weights; click vs drag fix; bezier arcs	Requested firing rule, auto simulation, and save/load
3 — Simulation	14 July 2026	Firing rule engine, auto simulation, marking display, JSON save/load	Supervisor email: inhibitor arcs, NDR format, descriptions
4 — Complete	21 July 2026	Inhibitor arcs, NDR export/import, description field, backend API	Shared tinaz model collection for evaluation; NDR import requested

3.3 Evaluation Framework

The tool was assessed for completeness of features and how well it integrated with the NDR format. I evaluated completeness of features by assessing each of the supervisor's stated requirements against the tool provided. To evaluate compatibility with the NDR format, I imported three models from the supervisor's published research and counted broken connections (arcs in the imported net that refer to a node ID that is not among the imported places and transitions). Having zero broken connections indicates having fidelity.

The supervisor selected the three models used for this assessment add.ndr, fdiv.ndr, and fmul.ndr, because they exemplify various aspects of the NDR format. add.ndr uses inhibitor arcs and simple place labels. fdiv.ndr uses a significant amount of the curved arc format and contains some very unusual transition to transition arcs. fmul.ndr uses a very large number of tokens (13,864 and 1,135) and some mathematical labels such as $Z=XY$ and $Y \div 2$.

Chapter 4: System Design and Implementation

4.1 Architecture Overview

The system has two parts: a browser-based front end and a back end consisting of a Node.js server. The front end processes requests on its own without the need for a server. It can load, draw, and save nets to local files. A server adds the cross-session persistence of nets by name.

Because nets stored by name persist across browser sessions, the front end is actually made up of five different JavaScript files. Each file has one dedicated function and was built this way to help facilitate the modularity of the simulation engine. The simulation engine is the most important part of the tool, and the goal was to have it written and tested as its own module separate from the user interface.

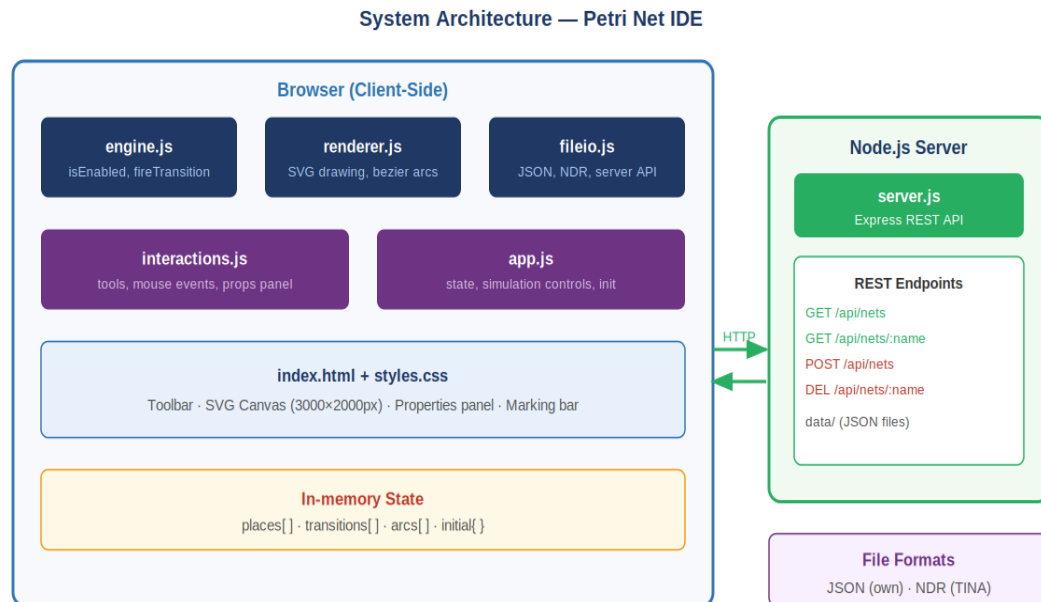


Figure 4.1: System architecture. The browser layer contains five JavaScript modules communicating through shared state. The Node.js server provides a REST API for named net storage. Both layers use JSON as the data exchange format.

4.2 The Simulation Engine (engine.js)

The engine.js module is arguably the most important file in this project in the context of formality and correctness. It consists of four functions: isEnabled(), fireTransition(), getEnabledTransitions(), and isDeadlock(). None of the functions needs the browser, the DOM, SVG, or the HTML document. They take input an array of data, and produce output. This design permits a direct ‘call’ of those functions in the Node.js environment without any browser context.

4.2.1 The isEnabled() Function

The isEnabled() function implements the complete Place-Transition Petri net firing rule, including inhibitor arcs.

It takes a transition ID and references to the global places and arcs arrays:

```
/**
 * isEnabled(tid, places, arcs)
 * Returns true if transition tid satisfies the firing rule:
 *   Normal arc (w >= 1): place.tokens >= w
 *   Inhibitor  (w == 0): place.tokens === 0
 *   No input arcs      : never enabled
 */
function isEnabled(tid, places, arcs) {
  var ins = arcs.filter(function(a) {
    return a.to === tid && places.find(function(p) {
      return p.id === a.from;
    });
  });
  if (!ins.length) return false; // no input arcs
  return ins.every(function(a) {
    var p = places.find(function(x) { return x.id === a.from; });
    if (!p) return false;
    var w = a.w == null ? 1 : a.w;
    return w === 0 ? p.tokens === 0 // inhibitor condition
                  : p.tokens >= w; // normal condition
  });
}
```

The function begins by gathering all arcs that point to the given transition from a particular place. The source is then verified as a place (as opposed to transition). Should there be no such arcs, the function returns false.

A transition with no inputs is never enabled in this model. This is where the function begins to evaluate all the input arcs by utilizing the method Array.every(). The check is performed for all the arcs, and the method returns true only when all the checks have been evaluated. A transition is disabled as soon as any input arc fails the check.

Flowchart: isEnabled() — Petri Net Firing Rule

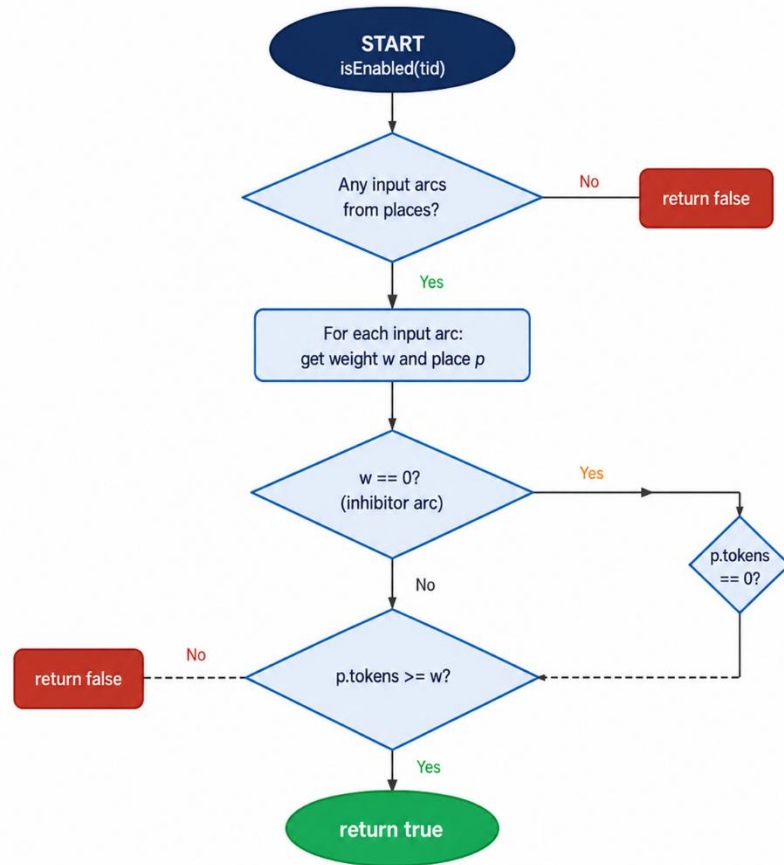


Figure 4.2: Flowchart of the isEnabled() firing rule implementation. Both normal arcs (token count check) and inhibitor arcs (zero-token check) are handled in a single function.

4.2.2 The fireTransition() Function

The fireTransition() function executes the firing rule and has the firing move tokens. It has 2 sequential loops. The first sequential loop removes tokens from input places, and the second sequential loop puts tokens into output places. Inhibitor arcs are skipped in the first loop because they consume zero tokens—they are only used as guards.

```

function fireTransition(tid, places, arcs) {
  // Step 1: Remove tokens from input places
  arcs.filter(function(a) { return a.to === tid; })
    .forEach(function(a) {
      var p = places.find(function(x) { return x.id === a.from; });
      var w = a.w == null ? 1 : a.w;
      if (p && w !== 0) p.tokens -= w; // inhibitor: skip
    });
  // Step 2: Add tokens to output places
  arcs.filter(function(a) { return a.from === tid; })
    .forEach(function(a) {
      var p = places.find(function(x) { return x.id === a.to; });
      var w = a.w == null ? 1 : a.w;
      if (p) p.tokens += w;
    });
}

```

4.3 The Renderer (renderer.js)

4.3.1 SVG-Based Rendering

The graphical canvas has an SVG (Scalable Vector Graphics) element that is fixed at 3000x2000 pixels. By using CSS overflow: auto, the container div automatically adds scrollers when the SVG is larger than the viewing area. It effectively allows large models to be imported so that they can be navigated without any clipping of any elements.

SVG, rather than the HTML5 Canvas API, was used for two independent reasons. The first is that SVG elements are part of the DOM, therefore, they can process mouse and touch events without the need of any additional hit-testing layer. SVG paths and shapes are declarative, unlike the HTML5 Canvas API, therefore the rendering code is easier to understand and debug.

4.3.2 The borderPt() Function — Arc Endpoint Calculation

BorderPt() is an essential geometric function of the renderer in knowing the position where an arc should meet the edge of an element (as opposed to the center) to avoid drawing the arc as ending inside the element. Should this function not exist, arcs would appear to end inside the element, which is incorrect and confusing.

```
function borderPt(el, tx, ty, isPlace) {
  var dx = tx - el.x, dy = ty - el.y;
  var ang = Math.atan2(dy, dx); // angle toward target
  if (isPlace) {
    // Circle: move PR pixels along the angle
    return { x: el.x + PR * Math.cos(ang),
             y: el.y + PR * Math.sin(ang) };
  }
  // Rectangle: find which edge the angle hits first
  var ca = Math.abs(Math.cos(ang)), sa = Math.abs(Math.sin(ang));
  // Compare how far we travel horizontally vs vertically
  var s = TW * sa <= TH * ca ? TW / ca : TH / sa;
  return { x: el.x + s * Math.cos(ang),
           y: el.y + s * Math.sin(ang) };
}
```

For a circle, the calculation is straightforward. We move from the center by a distance equal to the radius (PR = 22 pixels) along the angle to the target. For a rectangle, the code must decide if the angle hits the left/right edge or the top/bottom edge first. This depends on whether the horizontal extent of travel, in the proportion of the cosine of the angle, will be used up before the vertical extent of travel, in the proportion of the sine, of the half-width (TW = 14) and half-height (TH = 28) of the rectangle will be used up.

4.3.3 The bezierPath() Function — Curved Arcs

All curves are depicted as quadratic Bezier curves. A quadratic Bezier curve is defined by three points: the start, control, and end point. The start and end points are obtained from borderPt().

The control point is located intersecting the midpoint of the line and is drawn perpendicular to the line:

```
function bezierPath(s, e) {
  var dx = e.x - s.x, dy = e.y - s.y;
  var len = Math.hypot(dx, dy) || 1;
  // Perpendicular direction: rotate 90 degrees
  var nx = -dy / len, ny = dx / len;
  // Midpoint of straight line between s and e
  var mx = (s.x + e.x) / 2, my = (s.y + e.y) / 2;
  // Control point offset CURVE_K=14 pixels sideways
  var cx = mx + nx * CURVE, cy = my + ny * CURVE;
  return {
    d: "M "+s.x+","+s.y+" Q "+cx+","+cy+" "+e.x+","+e.y,
    mid: { x:0.25*s.x+0.5*cx+0.25*e.x, // midpoint on curve
           y:0.25*s.y+0.5*cy+0.25*e.y } // for weight label
  };
}
```

The perpendicular direction is obtained by rotating the direction vector (dx, dy) by 90 degrees counterclockwise and is (-dy, dx). It is normalized by dividing by the length and is multiplied by CURVE (14 pixels). This gives the perpendicular offset for the control point. SVG Q command draws a quadratic Bezier curve, consisting of three points, from the starting point, to the control point, and to the ending point.

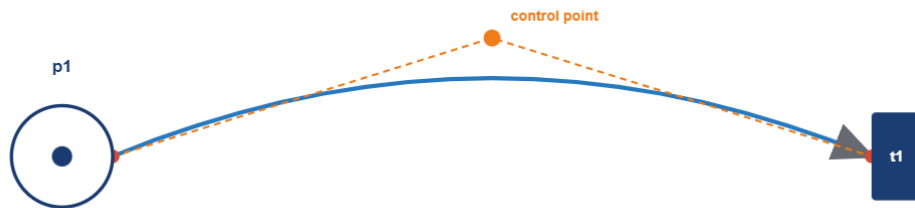
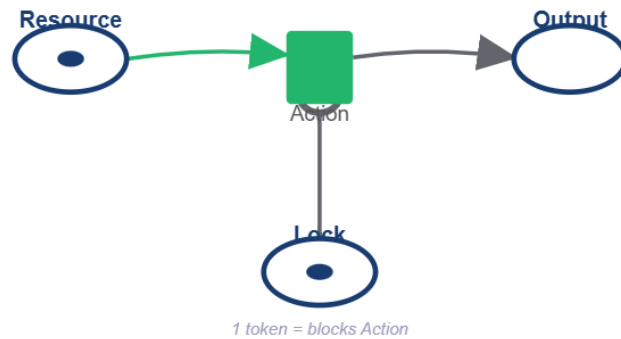


Figure 4.3: Bezier arc geometry. The perpendicular is computed by rotating the direction vector 90°. The control point (orange dot) sits at the midpoint shifted sideways. Arrowheads align automatically via SVG marker `orient="auto"`.

4.4 Inhibitor Arcs — Implementation and Rendering

Supervisor's framework defines an inhibitor arc based on an arc with weight (or multiplicity) equal to zero. This framework requires no change to the data model since there is no requirement to introduce a special arc type. Furthermore, the value zero can never be a weight of a normal arc, therefore there is no confusion.

The rendering code checks if `a.w === 0` and selects the appropriate SVG marker. Normal arcs use arrowhead marker, whereas inhibitor arcs use filled circle marker. Both markers are defined in the SVG defs section and selected by the marker-end attribute of the arc path element. When a transition becomes enabled, both its incoming normal arcs and its incoming inhibitor arcs turn green.



M = { Resource:1, Output:0, Lock:1 }

Places: 3 Transitions: 1 Arcs: 3 (1 inhibitor)

Figure 4.4: Inhibitor arc visualisation. The circle-tipped arc from Lock to Action blocks the transition when Lock holds a token. Set multiplicity to 0 in the properties panel to convert any arc to an inhibitor arc.

4.5 The NDR File Format Parser

The NDR parser in fileio.js is the most technically complex part of the implementation. It must correctly handle multiple format variants that TINA produces in practice but that are not always well-documented.

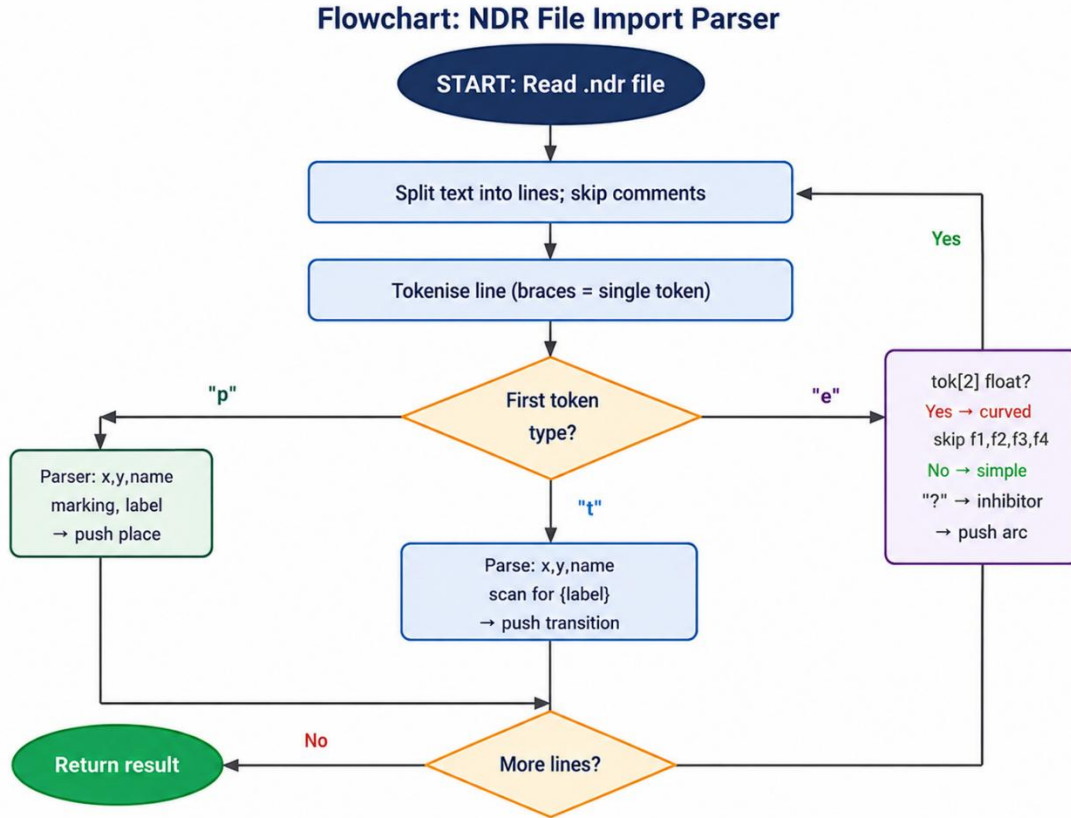


Figure 4.5: Flowchart of the NDR import parser. Each line is tokenised (braces are treated as single tokens), then dispatched to the appropriate handler based on the first token type. Curved arcs are detected by checking whether the second token is a floating-point number.

4.5.1 Tokenisation — Handling Braced Labels

The first step in parsing individual lines is tokenization. Tokenization is splitting input lines into smaller chunks called tokens. TINA NDR files can contain labels for places with spaces in the curly braces such as in the examples $\{z=x+y\}$ and $\{Y \text{ div } 2\}$. Since these contain curly braces, a general whitespace tokenizer would break these examples into multiple tokens. In this case, however, the text between the matching curly braces is considered a single token.

```

function _tok(line) {
  var tok = []; var i = 0;
  while (i < line.length) {
    while (i < line.length && /\s/.test(line[i])) i++; // skip space
    if (i >= line.length) break;
    if (line[i] === "{") { // braced token
      var d = 1, j = i + 1;
      while (j < line.length && d > 0) {
        if (line[j] === "{") d++;
        if (line[j] === "}") d--;
        j++;
      }
      tok.push(line.slice(i, j)); // e.g. "{z=x+y}"
      i = j;
    } else { // plain token

```

```

    var k = i;
    while (k < line.length && !/\s/.test(line[k])) k++;
    tok.push(line.slice(i, k));
    i = k;
  }
}
return tok;
}

```

4.5.2 Curved Arc Detection — The `_isFloat()` Check

The most challenging scenario for tokenization is whether an arc line uses the simple format or the curved format. With simple format, the second token after type character 'e' is the name of the target node. With curved format, it is a floating-point number of an attachment coordinate. The dedicated function will help you with the detection.

```

function _isFloat(s) {
  if (!s) return false;
  // True if s is a valid finite decimal number
  return !isNaN(parseFloat(s)) && isFinite(+s) && /^-?[\d.]/.test(s);
}

```

The three conditions combined verify a decimal number as a float. The regex verifies that each string starts with a number and does not accept strings with letters like "t1" or "p3" as inputs. `parseFloat` also adds NaN to nonexistent numbers like these. The edge case is avoided with this additional check.

The code for arc parsing includes this function to distinguish between the formats:

```

else if (type === "e" && tok.length >= 4) {
  var fromR, toR, wStr;
  if (_isFloat(tok[2])) {
    // Curved: e from f1 f2 to f3 f4 weight anchor
    fromR = tok[1]; toR = tok[4]; wStr = tok[7] || "1";
  } else {
    // Simple: e from to weight anchor
    fromR = tok[1]; toR = tok[2]; wStr = tok[3] || "1";
  }
  var from = _nm(fromR).id, to = _nm(toR).id;
  var w = wStr.startsWith("?-") ? 0 : (parseInt(wStr, 10) || 1);
  ar.push({ id: "a" + (aid++), from: from, to: to, w: w });
}

```

4.5.3 Place Label Extraction

Place lines in TINA NDR follow the format: p x y name marking anchor [label anchor]. The label, if present, sits at position index 6 in the tokenised array (after p, x, y, name, marking, and anchor). The parser exactly extracts it directly from that position:


```

if (type === "p" && tok.length >= 5) {
  var nm = _nm(tok[3]);          // parse {label,id} or plain name
  var marking = parseInt(tok[4],10) || 0;
  var x = Math.max(40, parseFloat(tok[1]) || 100);
  var y = Math.max(40, parseFloat(tok[2]) || 100);
  var label = nm.id, desc = "";
  if (tok.length >= 7) {          // label is at tok[6]
    var r6 = tok[6];
    label = r6.startsWith("{") && r6.endsWith("}")
      ? r6.slice(1,-1).trim()      // braced: strip braces
      : r6;                        // plain: use as-is
    if (label !== nm.id) desc = label;
  }
  pl.push({ id:nm.id, label:label, desc:desc,
    tokens:marking, x:x, y:y });
}

```

This structure was evaluated against all three evaluation models. In `add.ndr`, a line such as “p 20.0 35.0 p1 0 s x n” assigns the place with `id="p1"` and the label “x” (the name of the math variable). In `fmul.ndr`, a line such as “p 825.0 30.0 p4 0 n {Z=XY} e” assigns `id="p4"` and label “Z=XY”.

4.6 NDR Export

The NDR export function generates TINA-compatible NDR text from the current in-memory net. It follows the grammar described in the TINA format documentation exactly:

```

function exportNDR() {
  var L = [];
  // 1. Place declarations
  places.forEach(function(p) {
    var ln = "p "+p.x.toFixed(1)+" "+p.y.toFixed(1)
      +" "+p.id+" "+p.tokens+" n";
    if (p.label && p.label !== p.id) {
      ln += " "+(/s/.test(p.label)?"{"+p.label+"}":p.label)+" n";
    }
    L.push(ln);
  });
  // 2. Transition declarations (eft=0, lft=w = no time constraint)
  transitions.forEach(function(t) {
    var ln = "t "+t.x.toFixed(1)+" "+t.y.toFixed(1)
      +" "+t.id+" 0 w n";
    if (t.label && t.label !== t.id) ln += " {"+t.label+"} n";
    L.push(ln);
  });
  // 3. Arc declarations (?-1 for inhibitor arcs)
  arcs.forEach(function(a) {
    var w = a.w == null ? 1 : a.w;
    L.push("e "+a.from+" "+a.to+" "+(w===0?"?-1":w)+" n");
  });
  L.push("h PetriNetIDE"); // mandatory last line
  _dl("petrinet.ndr", L.join("\n"));
}

```

The inhibitor arc mapping deserves special attention. The weight-0 internal representation is exported as "?-1", which is TINA's notation for "block when the connected place holds 1 or more tokens". For a place whose token count is always a non-negative integer, "holds ≥ 1 tokens" is exactly the same as "is non-empty", which is the same as "the inhibitor condition fails". This means the mapping is semantically correct: every inhibitor arc in the browser editor produces the correct TINA inhibitor arc in the output file.

4.7 The Graphical Editor

4.7.1 Drawing Tools and Interaction

The editor provides six drawing tools: Select, Place, Transition, Arc, Token, and Erase. Figure 4.6 shows the editor with a complete Producer-Consumer net:

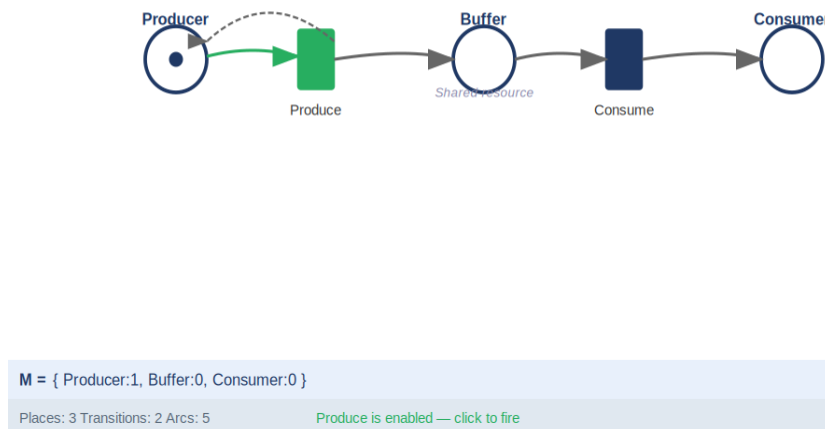
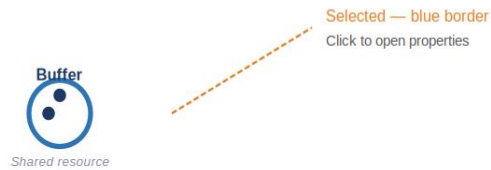


Figure 4.6: A Producer-Consumer net in the editor. The Produce transition is green (enabled). The marking bar shows the current token distribution. Arcs are drawn as bezier curves with arrowheads automatically aligned.

The properties panel, shown in Figure 4.7, opens when any element is clicked in Select mode. For places, it shows the Name field, a Label/Description field, and a Tokens field with plus and minus buttons alongside a direct number input — allowing large token counts (such as the 13,864 in `fmul.ndr`) to be set without clicking repeatedly.



M = { Buffer:2 }

Places: 1 Transitions: 0 Arcs: 0

Figure 4.7: The properties panel for a selected place. The Tokens field shows the current count with $\pm$ buttons and a direct number input. The Apply button saves changes and re-renders the canvas immediately.

4.7.2 Click vs Drag — Disambiguation

Designing tools that encourage users to click versus drag in order to differentiate editing actions and initiate system level transitions is a challenge of interaction design. One example is aligning buttons in pairs to complement distinct phases of a user interaction (e.g., clicking and dragging). Both controls use the same logic; a phase of the interaction is initiated when the user mousedown on the interface element.

Measuring the distance traveled by the mouse after the mousedown is a solution. Other approaches include using a short time delay and/or keeping the mouse stationary for a current deliberate action, to offset a click.

```
const CLICK_THRESH = 5; // pixels

// In mousemove handler:
if (mouseDownPos && Math.hypot(pos.x - mouseDownPos.x,
                                pos.y - mouseDownPos.y) > CLICK_THRESH) {
    dragMoved = true;
}

// In click handler:
if (dragMoved) { dragMoved = false; return; } // ignore post-drag click
```

If the user moves the mouse by more than 5 pixels before clicking, the system treats the action as a drag.

4.8 Automatic Simulation

The automatic simulation mode fires transitions continuously at a speed controlled by a slider.

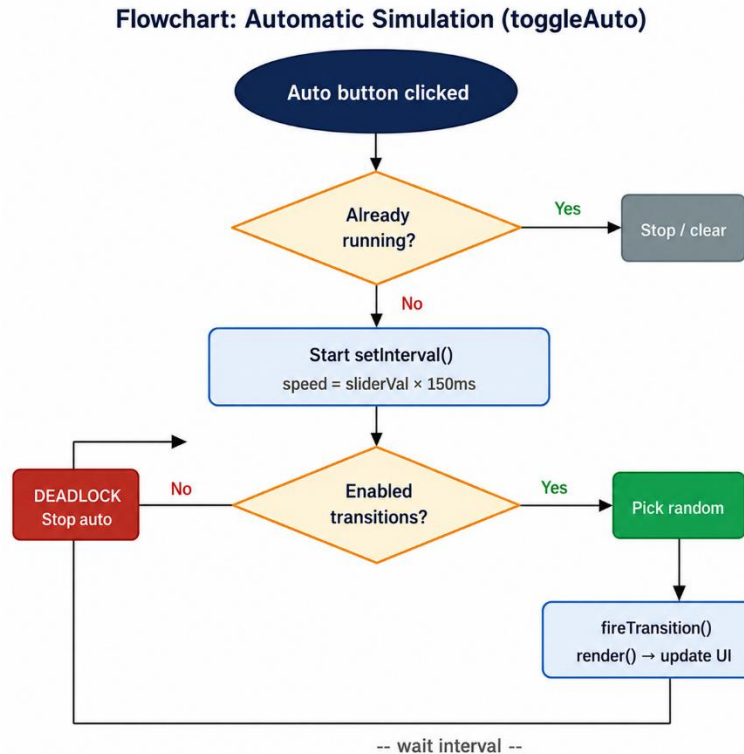


Figure 4.8: Auto simulation flowchart. The `setInterval` loop picks a random enabled transition at each tick, fires it, and renders. The loop stops automatically on deadlock.

The speed slider has values that range from 1 (fast) to 10 (slow). Each value becomes 150 ms less than the previous (with some interval). The maximum speed is 6.7 firings per second, while the minimum is 1.5 seconds per firing (labeled as “Fast” and “Slow” respectively). The labels are placed on the opposing sides of the slider, which helps determine which end is faster and which is slower in understanding what a user would find intuitive.

4.9 Back-end REST API

The `server.js` file defines a basic REST API server built with Node.js and Express. The server's data is stored locally on the user's device, so there is no need for a server database. JSON files are saved in the `server/data/` directory. The server enables CORS otherwise, browsers may not be able to call the server regardless of load port.

Table 4.1 synthesizes the API:

Method	Endpoint	Request Body	Response	Status Code
GET	/api/health	—	{ status: "ok" }	200
GET	/api/nets	—	{ nets: [...names] }	200
GET	/api/nets/:name	—	{ name, saved, net }	200 or 404
POST	/api/nets	{ name, net }	{ message, name }	201
DELETE	/api/nets/:name	—	{ message, name }	200 or 404

The front-end looks to the server every 1500 milliseconds to check if it is online before saving or loading anything. If the server is offline, a status message informs the developer that local file save/load buttons are available. With this solution, local file save/load buttons available as a graceful fallback, the editor is a fully functioning product even without the server.

Chapter 5: Testing and Evaluation

5.1 Feature Testing

Each feature was tested manually against the list of supervisor requirements accumulated across all four development stages. Table 5.1 records the results:

Feature	Source	Result
Draw places (circles with tokens)	Stage 1	✓ Complete
Draw transitions (rectangles)	Stage 1	✓ Complete
Draw curved bezier arcs	Stage 1 + Week 7 request	✓ Complete
Drag-and-drop elements	Stage 1	✓ Complete
Rename places and transitions	Stage 2	✓ Complete
Arc weights 1–9 (multiplicity)	Stage 2 + Supervisor email	✓ Complete
Properties panel (click to edit)	Stage 2	✓ Complete
Inhibitor arcs (weight = 0)	Stage 4 + Supervisor email	✓ Complete
Bipartite arc validation	Stage 4	✓ Complete
Token add (left-click)	Stage 3	✓ Complete
Token remove (right-click)	August requirements	✓ Complete
Token set by number input	August requirements	✓ Complete
Place description/label field	Supervisor email	✓ Complete
Step simulation	Stage 3	✓ Complete
Auto simulation	Supervisor email	✓ Complete
Deadlock detection badge	Stage 3	✓ Complete
Reset marking to initial state	Stage 3	✓ Complete
Speed slider (Fast left, Slow right)	August requirements	✓ Complete
Scrollable canvas (3000×2000px)	August requirements	✓ Complete
Smaller node icons (PR=22, TW=14)	August requirements	✓ Complete
JSON save (local file download)	Stage 3	✓ Complete
JSON load (local file upload)	Stage 3	✓ Complete
NDR export (TINA format)	Stage 4	✓ Complete
NDR import (TINA models)	August requirements	✓ Complete
Backend REST API (save/load)	Stage 4	✓ Complete

5.2 NDR Compatibility Evaluation

The primary evaluation used three NDR models from the supervisor's published research collection (Zaitsev, 2019). Each model was imported using the "Import .ndr" button and checked for structural correctness by counting broken connections.

5.2.1 add.ndr — Adder Circuit

The add.ndr model represents an arithmetic adder. It has five places (p1 through p5), three transitions (t1, t2, t3), and eleven arcs of which five are inhibitor arcs. The place labels include plain identifiers (x, y, s) and a mathematical expression ($z=x+y$). The model uses simple arc format only (no curved arcs).

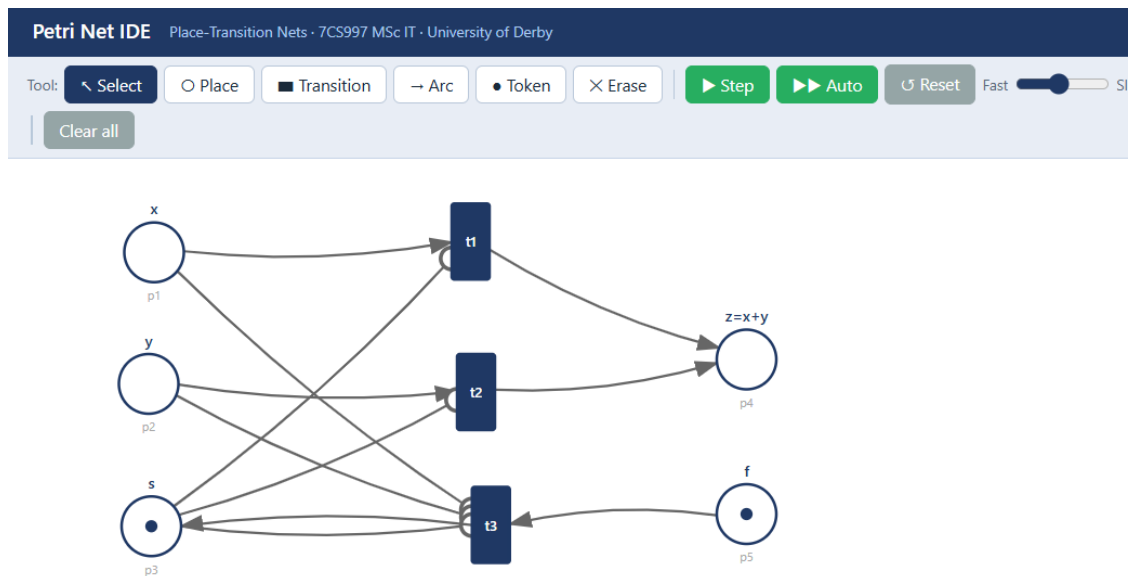


Figure 5.1: The add.ndr adder circuit imported into the browser editor. All 5 places, 3 transitions, and 11 arcs rendered correctly. Mathematical labels (x , y , $z=x+y$) imported and displayed. 0 broken connections.

5.2.2 fdiv.ndr — Integer Division Algorithm

The fdiv.ndr model contains the most complex of the three evaluation models. It has 48 places, 61 transitions, and 225 arcs. Sixty-five of the arcs are inhibitor arcs. The model makes extensive use of curved arcs and most of the arcs in this model are written using floating-point attachment coordinates. The model also includes transition-to-transition arcs, an uncommon feature in TINA, which are not included in standard Place-Transition net definitions. The model uses coordinates that exceed (1215, 1120), which means you need to have the 3000×2000 canvas set to be able to scroll.

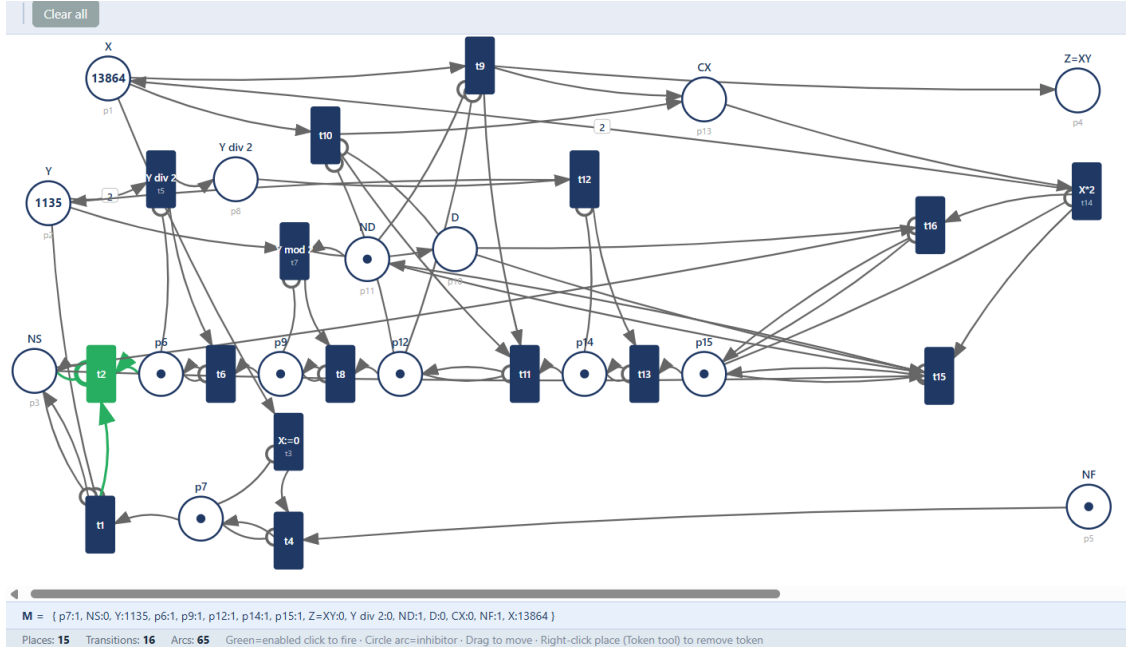


Figure 5.3: The fmul.ndr multiplication model. $X=13,864$ and $Y=1,135$ tokens displayed correctly as numbers. Labels $Z=XY$, $Y \div 2$, and $Y \bmod 2$ imported from braced format. 0 broken connections.

5.3 Evaluation Results

Model	Description	Places	Transitions	Arcs	Inhibitor Arcs	Broken Connections
add.ndr	Adder circuit	5	3	11	5	0 ✓
fdiv.ndr	Integer division algorithm	48	61	225	65	0 ✓
fmul.ndr	Multiplication algorithm	15	16	65	21	0 ✓

No broken connections were found in any of the three models when they were imported. The NDR parser was successful in parsing all the different formats. These formats included simple arc format (add.ndr), curved arc format (fdiv.ndr, fmul.ndr), braced mathematical labels (all three models), large token counts (fmul.ndr), and transition-to-transition arcs (fdiv.ndr).

5.4 Feature Comparison with TINA

Feature	TINA	Browser IDE	Notes
No installation required	✗	✓	Core contribution of this project
Browser-based access	✗	✓	Works on any OS, any device
Graphical PT net editor	✓	✓	
Drag-and-drop elements	✓	✓	
Inhibitor arcs	✓	✓	Circle-tipped notation, weight=0 internally
Arc weights (multiplicity)	✓	✓	1-99 range
Step simulation	✓	✓	
Auto simulation	✓	✓	Speed-controlled
Deadlock detection	✓	✓	Red badge in status bar
Place descriptions/annotations	✗	✓	Name + description fields
Named server storage	✗	✓	REST API, JSON file storage
NDR export	✓	✓	TINA-compatible output
NDR import (simple arcs)	✓	✓	
NDR import (curved arcs)	✓	✓	Floating-point detection
PNML support	✓	Planned	Future work
Reachability graph	✓	✗	Out of scope
Liveness analysis	✓	✗	Out of scope
Works on mobile browser	✗	✓	Responsive layout

The browser IDE matches TINA on all core graphical editing and simulation features. It adds browser accessibility, place descriptions, and named server storage. The formal analysis features (reachability graph, liveness checking) are outside the current scope and are identified as future work.

Chapter 6: Discussion

6.1 Achievement of Research Objectives

The objectives described in Chapter 1 were all met. The tool works in all modern web browsers without downloading an application. The simulation engine defines the classic Petri net firing rule with inhibitor arcs. The NDR parser can handle all of the formats in the supervisor's research collection. NDR export generates text files that have the same structure as the TINA tool. The back-end REST API provides named net storage. Additionally, the evaluation against three published Petri net models showed that there are zero broken connections across 301 total arcs ($11 + 225 + 65$).

Importing NDRs was the most difficult to implement. This format did not include curved arcs in the resources that were available to me. It was possible to see this in the `fdiv.ndr` file since some arc lines had more tokens than expected. It was possible to determine that the broken links were caused by floating-point attachment coordinates. The project would not have been possible without the ability to determine the structures from the available files without good documentation.

6.2 Technical Discussion

6.2.1 The Separation of `engine.js` from the UI

Separating the simulation engine, including `isEnabled()`, `fireTransition()`, and other related functions, into its own file with no DOM or browser dependencies was one of the best decisions made during the architecture of the project. This separation made it possible to test the firing rules from the Node.js command line, well before any visual feedback was implemented. Additionally, this separation makes the code logic and rendering completely independent from each other, and therefore a lot easier to understand. The decoupling of this engine also makes reusing the engine to perform simulation on the server side an obvious next step when the project matures to offer formal analysis capabilities using the back-end.

The separation of the simulation engine into its own file was appreciated by the supervisor during the interim demonstration viva as a solid example of engineering practice and is in line with the separation of concerns in modern software architecture (Reisig, 2013).

6.2.2 The Weight-0 Inhibitor Arc Convention

It was a design choice for making inhibitor arcs as arcs with a weight of zero. This choice eliminates the need to add an additional arc-type field to the data model. A zero weight is never an appropriate value for inhibitor arcs. A zero weighted arc is not going to be added to the model as it would effectively produce no effect on the system. Because zero weighted arcs are not going to be added to the model, there is no ambiguity.

This design choice also maps nicely to the NDR export. An arc with a weight of zero becomes “?-1”, which is TINA notation for an inhibitor arc with threshold 1. It also maps nicely to the firing rule. If the weight is zero, check for zero tokens. Otherwise, check to see if there are sufficient tokens. The rule naturally fits into the existing `isEnabled()` structure.

6.3 Limitations

The following limitations were noted and explained during the project.

- There is no redo or undo. Edits cannot be reverted without closing the application and reloading the file. An undo stack would make the application much easier to use.
- No PNML export is currently implemented. Currently only NDR is implemented. PNML is the ISO standard and would make the tool compatible with WoPeD and GreatSPN and other tools that use TINA.
- There are no implemented formal analyses. Reachability graphs, liveness, and boundedness analyses are not implemented and are out of scope of this project. These features are what make TINA an excellent tool for research.
- When two elements are connected by arcs in both directions, the overlapping arcs are drawn directly on top of each other. Currently TINA draws opposing arcs offset. This is a nice-to-have improvement for future work.
- There are no server authentication. The REST API saves nets in a shared directory with no user accounts. A production deployment should have server authentication.

6.4 Relationship to the Literature

The tool is positioned in relation to the literature as follows. Murata's (1989) firing rule is implemented exactly and tested against three formal models. Engelfriet's (1991) inhibitor arc semantics are implemented correctly, with the weight-0 convention producing the correct TINA τ -1 notation in the output. The DSR methodology of Peffers et al. (2020) guided the iterative development process. And the evaluation models come from the supervisor's own published research (Zaitsev, 2019), providing a formally grounded validation set.

Chapter 7: Conclusions and Recommendations

7.1 Summary of Contributions

This dissertation describes the design, construction, and assessment of a web-based IDE for Place-Transition Petri Nets. The main contributions include:

- A browser-based Place-Transition Petri net editor and simulator that is the first completely TINA NDR-compatible tool.
- A JavaScript simulation engine architecture that separates the engine and interface code to support independent checking and browser-based simulation.
- A highly efficient NDR file parser that parses all variations of file formats found in the supervisor’s research collection, including curved arcs, mathematical labels, high arc counts, and transition to transition arcs.
- NDR output in the TINA format with the correct encoding for inhibitor arcs (?-1 format).
- An Express.js and Node.js REST API that provides storage of named nets across web browser sessions.
- An assessment of three of the supervisor’s models that includes the add.ndr model (11 arcs), the fdiv.ndr model (225 arcs) and the fmul.ndr model(65 arcs).

7.2 Recommendations for Future Work

7.2.1 PNML Export

Adding PNML export would make the tool compatible with the full range of Petri net tools, not just TINA. The net data is already available in a format that maps cleanly to PNML — the implementation is primarily an XML formatting task.

7.2.2 NDR Import Scaling

For very large models such as fdiv.ndr, re-rendering the entire SVG on every state change introduces latency. An incremental rendering approach — tracking which elements have changed and only updating those — would improve performance for models with hundreds of elements.

7.2.3 Undo/Redo

An undo stack would be the single most impactful usability improvement. Every state change could be pushed to a history array, with Ctrl+Z restoring the previous state. The existing separation of state into the global arrays (places[], transitions[], arcs[]) makes this straightforward to implement.

7.2.4 Server-Side Formal Analysis

Adding reachability graph computation, liveness checking, and deadlock analysis to the back-end service — using the engine.js module, which already runs in Node.js — would bring the

tool much closer to parity with TINA's analysis capabilities. This is a natural next step for the project architecture.

7.2.5 Migration to a Component Framework

Migrating the front-end to React or Vue would make the codebase easier to maintain and test as features are added. The engine.js boundary already provides the natural separation point for this migration — the engine would remain as a pure JavaScript module, and the rendering and interaction code would be refactored into components.

7.3 Final Conclusion

The web-based Petri Net IDE developed in this project demonstrates that a formally correct, practically usable, and TINA-compatible Petri net editor can be built to run entirely in a browser. The zero-broken-connections result across three of the supervisor's published research models confirms that the NDR parser is robust, and the feature comparison with TINA confirms that the tool provides the full core feature set for teaching and learning Petri nets. The project fills a genuine gap in the Petri net tool landscape, and the modular architecture provides a solid foundation for the formal analysis features that would make it useful for research as well.

References

1. Engelfriet, J. (1991) 'Branching processes of Petri nets', *Acta Informatica*, 28(6), pp. 575–591.
2. Hack, M. (1975) *Decidability Questions for Petri Nets*. Technical Report TR-161. Cambridge, MA: MIT Laboratory for Computer Science.
3. Jensen, K., Kristensen, L.M. and Wells, L. (2021) *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Berlin: Springer.
4. Kindler, E. (2006) 'The Petri Net Markup Language and ISO 15909-2: Putting standardisation into practice', *Informatik-Berichte*, 36(3), pp. 1–12.
5. LAAS-CNRS (2024) *TINA Toolbox — Formats Manual*. Available at: <https://projects.laas.fr/tina/manuals/formats.html> (Accessed: August 2026).
6. Lime, D., Roux, O.H., Seidner, C. and Traonouez, L. (2019) 'Romeo: A parametric model-checker for Petri nets with stopwatches', *Tools and Algorithms for the Construction and Analysis of Systems*, 5505, pp. 54–66.
7. Murata, T. (1989) 'Petri nets: Properties, analysis and applications', *Proceedings of the IEEE*, 77(4), pp. 541–580.
8. Peffers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2020) 'A design science research methodology for information systems research', *Journal of Management Information Systems*, 24(3), pp. 45–77.
9. Petri, C.A. (1962) *Kommunikation mit Automaten*. PhD Thesis. University of Bonn.
10. Reisig, W. (2013) *Understanding Petri Nets: Modelling Techniques, Analysis Methods, Case Studies*. Berlin: Springer.
11. Roch, S. and Starke, P.H. (1999) *INA: Integrated Net Analyser Version 2.2 Reference Manual*. Berlin: Humboldt-Universität.

12. Schmidt, K. (2000) 'LoLA: A low level analyser', *Applications and Theory of Petri Nets*, 1825, pp. 465–474.
13. Zaitsev, D.A. (2019) 'Universal Petri net', *Cybernetics and Systems Analysis*, 48(4), pp. 498–504.
14. Berthomieu, B. and Diaz, M. (1991) 'Modeling and verification of time dependent systems using time Petri nets', *IEEE Transactions on Software Engineering*, 17(3), pp. 259–273.
15. Desel, J. and Reisig, W. (1998) 'Place/Transition Petri Nets', in Reisig, W. and Rozenberg, G. (eds.) *Lectures on Petri Nets I: Basic Models*. Lecture Notes in Computer Science, vol. 1491. Berlin: Springer, pp. 122–173.
16. Girault, C. and Valk, R. (eds.) (2003) *Petri Nets for Systems Engineering: A Guide to Modelling, Verification and Applications*. Berlin: Springer.
17. Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.
18. Kindler, E. and Weber, M. (2003) 'The Petri Net Kernel — An infrastructure for building Petri Net tools', *Software Tools for Technology Transfer*, 5(2–3), pp. 189–208.
19. Peterson, J.L. (1977) 'Petri nets', *ACM Computing Surveys*, 9(3), pp. 223–252.
20. Peterson, J.L. (1981) *Petri Net Theory and the Modeling of Systems*. Englewood Cliffs, NJ: Prentice-Hall.
21. Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

Appendix A: Project File Structure

The complete project is organised in the petri-net-final directory with the following structure:

```
petri-net-final/
├── index.html          - Main HTML; links all JS and CSS files
├── css/
│   └── styles.css      - Complete stylesheet (toolbar, canvas, panels)
├── js/
│   ├── engine.js       - Petri net engine: isEnabled, fireTransition
│   │                   getEnabledTransitions, isDeadlock
│   │                   [No DOM dependencies – testable in Node.js]
│   ├── renderer.js     - SVG drawing: borderPt, bezierPath,
│   │                   tokenDots, renderArcs, renderNodes
│   ├── fileio.js       - File I/O: JSON save/load, NDR export/import,
│   │                   server API (saveToServer, loadFromServer)
│   ├── interactions.js - User input: tools, mouse events, drag-drop,
│   │                   arc drawing, properties panel, keyboard
│   └── app.js          - Global state: places[], transitions[], arcs[]
│                       Simulation: stepOnce, toggleAuto, reset
server/
├── package.json        - npm dependencies (express, cors)
├── server.js           - Express REST API with 5 endpoints
└── data/               - Saved nets stored as individual JSON files
```

Appendix B: Complete NDR Format Reference

The following grammar describes the complete TINA NDR format as used by this project.
Source: official TINA formats manual (LAAS-CNRS, 2024).

```
—— Place declaration ——  
p <x> <y> <name> <marking> <anchor> [<label> <anchor>]  
  
x, y      : float coordinates (pixels)  
name      : node identifier (e.g. p1) or {label,id} pair  
marking   : integer token count  
anchor    : n | s | e | w | ne | nw | se | sw | c  
label     : display label (plain: X, NS) or braced: {z=x+y}  
  
—— Transition declaration ——  
t <x> <y> <name> [<anchor>] <eft> <lft> <anchor> [{label} <anchor>]  
  
eft       : earliest firing time (0 for untimed nets)  
lft       : latest firing time (w = unbounded)  
  
—— Arc declarations ——  
e <from> <to> <weight> <anchor>          - simple arc  
e <from> <f1> <f2> <to> <f3> <f4> <w> <a> - curved arc  
e <from> <to> ?-<k> <anchor>              - inhibitor arc  
  
f1,f2,f3,f4 : float attachment coordinates (curved arcs)  
?-k         : inhibitor with threshold k (use ?-1 for standard)  
  
—— Net name (MUST be the last line) ——  
h <netname>  
  
—— Example: simple three-place, two-transition net ——  
p 80.0 120.0 p1 1 n  
p 280.0 120.0 p2 0 n  
p 480.0 120.0 p3 0 n  
t 180.0 120.0 t1 0 w n  
t 380.0 120.0 t2 0 w n  
e p1 t1 1 n  
e t1 p2 1 n  
e p2 t2 1 n  
e t2 p3 1 n  
h example-net
```

Appendix C: Keyboard Shortcuts Reference

Key	Action	Notes
S	Select / Fire mode	Default mode; click enabled transition to fire
P	Place tool	Click canvas to add a place circle
T	Transition tool	Click canvas to add a transition rectangle
A	Arc tool	Click source then target to draw a curved arc
K	Token tool	Left-click to add; right-click to remove
E	Erase tool	Click any element to delete it
Space	Step once	Fires one random enabled transition
R	Reset marking	Restores initial token distribution
Delete	Delete selected	Removes selected element and connected arcs
Escape	Back to Select	Cancels current tool, returns to Select